

Explicación técnica — Bugs pendientes no corregidos en el repositorio nuevo

Para: Equipo de desarrollo — `opensir_oaxaca_backend` **De:** Alfonso Sandoval (asandoval), Infraestructura y Ciberseguridad **Fecha:** 3 de septiembre de 2026

Este documento explica en detalle, con código exacto, por qué dos bugs ya diagnosticados durante la estabilización post-corte (ver SIGOB-OAX-CORTE01 v2.2, secciones 13.8 y 13.9) siguen sin resolverse en el repositorio nuevo (`opensir_oaxaca_backend`, rama `main`), confirmado durante la migración del 3 de septiembre de 2026.

1. `opensir/recaudacion/views/cargo.py` — por qué bajar de 100 a 50 no arregla el timeout

Endpoint afectado

`POST /recaudacion/consulta-caja/padron/`

Vista: `cargos_por_padron`, dentro de `CargoViewSet` (línea ~742 en el checkout de producción).

El cambio que sí se aplicó en el repo nuevo

```
- if timeout or len(cargos) > 100:  
+ if timeout or len(cargos) > 50:
```

Este es el **único** cambio relacionado con este endpoint en el repositorio nuevo. Reduce el umbral de cantidad de cargos a partir del cual la respuesta se delega a una tarea asíncrona de Celery (con polling desde el frontend), en vez de responder de forma síncrona.

Por qué esto no resuelve el problema — el orden de las operaciones

El código completo, en el orden real en que se ejecuta, es:

```
cargos = Cargo.objects.select_related().filter(  
    content_type__model=padron_obj.__class__.__name__.lower(),  
    object_id=padron_obj.id,  
    saldado=False,  
    entidad_id=request.user.entidad.id,  
    tipo_de_cargo__canales_de_pago=canal_de_pago,  
    eliminado=False  
) .exclude(estados_globales_id=5)  
  
recalcular_adeudos_de_cargos(cargos)          # <- PASO 1: siempre se ejecuta,  
                                              # sin importar el volumen  
  
cargos = Cargo.objects.select_related().filter(  
    content_type__model=padron_obj.__class__.__name__.lower(),  
    object_id=padron_obj.id,  
    saldado=False,  
    entidad_id=request.user.entidad.id,  
    tipo_de_cargo__canales_de_pago=canal_de_pago,  
    eliminado=False  
) .exclude(estados_globales_id=5)
```

```
# ... misma consulta repetida ...
).exclude(estados_globales_id=5)

if timeout or len(cargos) > 50:          # <- PASO 2: la decisión de ir
    # delega a Celery                    # a Celery llega DESPUÉS
else:
    # procesa síncrono
```

El recálculo pesado (recalcular_adeudos_de_cargos) ocurre en el PASO 1, antes de que el código sepa cuántos cargos hay. El `if` que decide si delegar a Celery está en el PASO 2, después de que ya se pagó el costo completo del recálculo.

Cambiar el número de 100 a 50 solo mueve el punto de corte de una decisión que ocurre **después** del trabajo pesado. No cambia en absoluto cuánto tarda el PASO 1, que es donde se concentra el tiempo cuando un padrón tiene muchos años de adeudo acumulado (el caso observado: predio con historial extenso, timeout de decenas de segundos a varios minutos).

Analogía simple

Es como decidir si vas a pedir ayuda para cargar unas cajas **después** de ya haberlas cargado tú solo. Bajar el número de “a partir de cuántas cajas pido ayuda” no sirve de nada si ya cargaste todas antes de hacer la pregunta.

Qué se necesitaría para resolverlo de fondo

Mover la evaluación de cantidad de cargos **antes** del recálculo, usando una consulta ligera de conteo (`Cargo.objects.filter(...).count()`, sin traer los objetos completos ni recalcular nada), y decidir ahí mismo si: - Hay pocos cargos → recalcular síncrono y responder de inmediato (como hoy). - Hay muchos cargos → delegar **todo el trabajo**, incluyendo el recálculo, a la tarea de Celery (`consulta_adeudos_task`), no solo la construcción de la respuesta final.

Esto es coherente con el patrón que el propio repositorio ya usa correctamente en `cargos_por_folio`, donde sí se cuenta primero (`total_cargos = ...count()`) y se decide antes de hacer trabajo pesado.

Por qué la tarea nunca aparece en la cola de Celery

Este mismo bug explica un síntoma adicional observado: al monitorear la cola de Celery durante un timeout de un padrón con historial extenso, **no aparece ninguna tarea nueva**, ni fallida ni en espera.

La línea que efectivamente encola el trabajo es:

```
if timeout or len(cargos) > 50:
    task = consulta_adeudos_task.apply_async(
        args=(tipo_padron, padron_obj.id, canal_de_pago.id, request.user.id),
        task_id=id_task
    )
```

Esta línea vive en el PASO 2, **después** del recálculo síncrono del PASO 1. Si el recálculo tarda más de lo que el gateway (Cloudflare, nginx) está dispuesto a esperar, la conexión se corta con un 504 antes de que la

ejecución de Python llegue siquiera a esta línea — la tarea **nunca se despacha**, porque el código nunca alcanza a pedírselo a Celery.

No es un problema de Celery, del worker, ni del broker (Redis): el worker está sano y disponible, simplemente nunca recibe la solicitud para este caso.

Cómo confirmarlo en vivo: mientras se reproduce el timeout desde la Ventanilla (predio con historial extenso), monitorear tareas activas en BE3:

```
docker compose -f docker-compose.be03.yml exec celeryworker celery -A config.celery_app inspect active
```

Si el timeout ocurre y no aparece ninguna tarea `consulta_adeudos_task` nueva durante ni después del intento, se confirma que la petición murió en el recálculo síncrono sin llegar a tocar Celery en ningún momento.

2. `opensir/recaudacion/utils/recargos_cargos.py` — bug de indentación (`UnboundLocalError`)

Función afectada

`registrar_recargo_en_cargos()`, alrededor de la línea 779–820 (numeración aproximada, puede variar levemente entre checkouts).

El código tal como está, en el repositorio nuevo (idéntico al de producción)

```
if tipo_de_recargo in recargos_existentes.values_list('tipo_de_recargo', flat=True):
    continue

if tipo_de_recargo.recalculable_manual and recalcular == False:
    continue

    is_already_applied = recargos_existentes.filter(
        tipo_de_recargo_id=tipo_de_recargo.id,
        cargo=cargo,
        estado_del_recargo_id=1 # Solo considerar los vigentes
    ).exists()

if is_already_applied:
    continue
```

Por qué esto es un error garantizado, no ocasional

En Python, si una variable se asigna **en cualquier parte** de una función, Python la trata como local a toda esa función — sin importar en qué rama de código esté esa asignación.

Mirando la indentación real: la línea que asigna `is_already_applied` está **dentro** del bloque del `if` de arriba

(if tipo_de_recargo.recalculable_manual and recalcular == False:), justo después de un continue. Esto la vuelve código inalcanzable en los dos únicos casos posibles:

- **Si la condición del if es verdadera:** el continue ya interrumpió el ciclo antes de llegar a esa línea.
- **Si la condición del if es falsa:** todo el bloque (incluida esa asignación) se salta por completo, porque está anidado dentro del if.

En ningún escenario posible se ejecuta esa asignación. Cuando el código llega, unas líneas más abajo, a if is_already_applied:, intenta leer una variable que Python considera local a la función pero que nunca fue asignada en esta ejecución — esto produce:

```
UnboundLocalError: local variable 'is_already_applied' referenced before assignment
```

Esto no es un caso raro o de borde: ocurre en el flujo normal de registro de recargos, cada vez que se procesa un tipo de recargo que no fue descartado por el primer if de la lista.

El fix — mover la indentación, sin cambiar la lógica

```
if tipo_de_recargo in recargos_existentes.values_list('tipo_de_recargo', flat=True):
    continue

if tipo_de_recargo.recalculable_manual and recalcular == False:
    continue

is_already_applied = recargos_existentes.filter(          # <- ya NO anidado
    tipo_de_recargo_id=tipo_de_recargo.id,              # dentro del if
    cargo=cargo,
    estado_del_recargo_id=1
).exists()

if is_already_applied:
    continue
```

El único cambio necesario es quitar la indentación extra de las 5 líneas de is_already_applied, para que queden al mismo nivel que el if anterior (fuera de él), no dentro. La lógica de negocio no cambia — solo se corrige dónde vive la línea dentro de la estructura del código.

Contexto adicional

Este mismo archivo ya tuvo un incidente relacionado documentado durante la instalación (INC01, cadena de bloqueos en recaudación — ver Memoria de Corte v2.2, sección 13.1.1): un UnboundLocalError de la misma naturaleza (variable asignada en código inalcanzable tras un continue) causó transacciones abiertas sin cerrarse, amplificadas por ATOMIC_REQUESTS=True. Aquel incidente fue corregido en su momento, pero un cambio posterior —el que introdujo la condición recalculable_manual— reintrodujo el mismo patrón de error en la misma función. Se sugiere que cualquier cambio futuro a esta función pase por una revisión de indentación especialmente cuidadosa, dado el antecedente.

Resumen para seguimiento

Archivo	Problema	Estado	Riesgo si no se corrige
views/cargo.py	Recálculo síncrono antes de decidir si delegar a Celery	Pendiente	Timeouts persistentes en consulta de adeudos para padrones con historial extenso
utils/recargos_cargos.py	Indentación incorrecta causa UnboundLocalError	Pendiente	Falla en el registro de recargos en el flujo normal, no solo en casos de borde